

Методичка 16. Декораторы, property и магические методы

1. Зачем нужна эта тема

Эта тема нужна, чтобы писать более аккуратные и удобные классы и функции.

До этого уже изучались:

- функции;
- классы;
- методы;
- инкапсуляция;
- наследование.

Теперь нужно понять несколько инструментов, которые часто встречаются в более взрослом Python-коде:

- декораторы;
- `@property` ;
- setter;
- магические методы;
- статические методы;
- классовые поля.

Эти инструменты помогают:

- добавлять дополнительное поведение к функциям;
- красиво получать данные объекта;
- контролировать изменение полей;
- удобно выводить объект на экран;
- хранить общие данные для всех объектов класса;
- писать код, который выглядит проще для пользователя класса.

Например, вместо такого кода:

```
cat.get_name()
```

можно сделать так:

```
cat.name
```

Но при этом внутри всё равно может выполняться проверка или дополнительная логика.

2. Главная идея темы

Главная идея этой темы: Python позволяет управлять поведением функций и объектов более гибко.

Декоратор — функция, которая принимает другую функцию и возвращает изменённую версию этой функции.

`@property` — способ сделать метод похожим на обычное поле.

Setter — метод, который срабатывает при изменении свойства.

Магический метод — специальный метод с двойными подчёркиваниями, например `__str__`.

`__str__` — метод, который отвечает за красивое текстовое представление объекта.

Статический метод — метод внутри класса, которому не нужен доступ к конкретному объекту через `self`.

Классовое поле — переменная, общая для всего класса, а не отдельная для каждого объекта.

3. Базовый синтаксис

Простой декоратор

```
def decorator(func):
    def wrapper():
        print("До вызова функции")
        func()
        print("После вызова функции")

    return wrapper

@decorator
def hello():
    print("Привет")

hello()
```

Что происходит:

- функция `hello()` передаётся в декоратор;
- декоратор возвращает новую функцию `wrapper` ;
- при вызове `hello()` сначала выполняется дополнительный код.

На что обратить внимание:

- декоратор пишется через `@` ;
- внутри декоратора обычно есть функция-обёртка;
- декоратор должен вернуть функцию, а не вызвать её сразу.

Декоратор с аргументами функции

```
def decorator(func):
    def wrapper(name):
        print("Функция начинается")
        func(name)
        print("Функция закончилась")

    return wrapper

@decorator
def hello(name):
    print("Привет, ", name)

hello("Аня")
```

Если исходная функция принимает параметры, `wrapper` тоже должен их принять.

`@property`

```
class Cat:
    def __init__(self, name):
        self.__name = name

    @property
    def name(self):
        return self.__name

cat = Cat("Барсик")

print(cat.name)
```

Метод `name()` вызывается как обычное поле `cat.name` .

Setter

```
class Cat:
    def __init__(self, name):
        self.__name = name

    @property
    def name(self):
        return self.__name

    @name.setter
    def name(self, new_name):
        if new_name != "":
            self.__name = new_name

cat = Cat("Барсик")

cat.name = "Мурзик"

print(cat.name)
```

Setter позволяет контролировать изменение значения.

Магический метод `__str__`

```
class Cat:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __str__(self):
        return f"Кот: {self.name}, возраст: {self.age}"

cat = Cat("Барсик", 3)

print(cat)
```

Когда объект передают в `print`, Python вызывает `__str__`.

4. Разбор темы от простого к сложному

Шаг 1. Функция-обёртка без декоратора

```
def hello():
    print("Привет")

def wrapper():
    print("До")
    hello()
    print("После")

wrapper()
```

Здесь `wrapper()` вручную вызывает `hello()` и добавляет действия до и после.

Шаг 2. Декоратор

```
def add_logs(func):
    def wrapper():
        print("Начало")
        func()
        print("Конец")

    return wrapper

@add_logs
def say_hello():
    print("Привет")

say_hello()
```

Теперь дополнительное поведение подключается через `@add_logs` .

Шаг 3. Декоратор для проверки

```
def check_email(func):
    def wrapper(email):
        if "@" in email:
            func(email)
        else:
            print("Некорректный email")

    return wrapper

@check_email
def register(email):
    print("Регистрация:", email)

register("test@mail.com")
register("wrong_email")
```

Декоратор может проверять данные перед вызовом функции.

Шаг 4. Проблема обычного getter

```
class User:
    def __init__(self, name):
        self.__name = name

    def get_name(self):
        return self.__name

user = User("Аня")

print(user.get_name())
```

Код работает, но вызов `get_name()` иногда выглядит громоздко.

Шаг 5. `@property`

```
class User:
    def __init__(self, name):
        self.__name = name

    @property
    def name(self):
        return self.__name

user = User("Аня")

print(user.name)
```

Снаружи это выглядит как обычное поле, но внутри работает метод.

Шаг 6. Setter с проверкой

```

class User:
    def __init__(self, age):
        self.__age = age

    @property
    def age(self):
        return self.__age

    @age.setter
    def age(self, new_age):
        if new_age >= 0:
            self.__age = new_age
        else:
            print("Возраст не может быть отрицательным")

user = User(15)

user.age = 16
user.age = -5

print(user.age)

```

Setter защищает объект от неправильного значения.

Шаг 7. Магический метод `__str__`

Без `__str__`:

```

class Product:
    def __init__(self, title, price):
        self.title = title
        self.price = price

product = Product("Книга", 500)

print(product)

```

Вывод будет техническим и неудобным.

С `__str__`:


```
class Product:
    def __init__(self, title, price):
        self.title = title
        self.price = price

    def __str__(self):
        return f"{self.title}: {self.price} руб."

product = Product("Книга", 500)

print(product)
```

Теперь объект выводится красиво.

Шаг 8. Статический метод

```
class MyMath:
    @staticmethod
    def get_square(number):
        return number * number

print(MyMath.get_square(5))
```

Статический метод не использует `self`.

Он логически относится к классу, но не зависит от конкретного объекта.

Шаг 9. Классовое поле

```
class Cat:
    count = 0

    def __init__(self, name):
        self.name = name
        Cat.count = Cat.count + 1

cat1 = Cat("Барсик")
cat2 = Cat("Мурка")

print(Cat.count)
```

`count` — общее поле класса. Оно считает, сколько объектов было создано.

5. Частые ошибки учеников

Ошибка 1. Декоратор вызывает функцию сразу

Неправильно:

```
def decorator(func):
    def wrapper():
        print("До")
        func()
        print("После")

    return wrapper()

@decorator
def hello():
    print("Привет")
```

Почему это ошибка:

Нужно вернуть функцию `wrapper`, а не результат её вызова.

Правильно:

```
def decorator(func):
    def wrapper():
        print("До")
        func()
        print("После")

    return wrapper
```

Ошибка 2. Забывают вызвать исходную функцию внутри `wrapper`

Неправильно:

```
def decorator(func):
    def wrapper():
        print("До")
        print("После")

    return wrapper
```

Почему это ошибка:

Исходная функция вообще не выполнится.

Правильно:

```
def decorator(func):
    def wrapper():
        print("До")
        func()
        print("После")

    return wrapper
```

Ошибка 3. `wrapper` не принимает параметры

Неправильно:

```
def decorator(func):
    def wrapper():
        func()

    return wrapper

@decorator
def hello(name):
    print(name)

hello("Аня")
```

Почему это ошибка:

Функция `hello` принимает параметр, значит `wrapper` тоже должен принять его.

Правильно:

```
def decorator(func):
    def wrapper(name):
        func(name)

    return wrapper
```

Ошибка 4. Путают `property` и обычный метод

Неправильно:

```
print(cat.name())
```

Почему это ошибка:

Если `name` оформлен через `@property`, его вызывают без скобок.

Правильно:

```
print(cat.name)
```

Ошибка 5. Делают setter без property

Неправильно:

```
class User:
    @age.setter
    def age(self, value):
        self.__age = value
```

Почему это ошибка:

Setter должен быть привязан к уже существующему `@property`.

Правильно:

```
class User:
    @property
    def age(self):
        return self.__age

    @age.setter
    def age(self, value):
        self.__age = value
```

Ошибка 6. `__str__` возвращает не строку

Неправильно:

```
def __str__(self):
    return self.price
```

Почему это ошибка:

`__str__` должен возвращать строку.

Правильно:

```
def __str__(self):  
    return str(self.price)
```

Ошибка 7. Используют self в staticmethod

Неправильно:

```
class MyMath:  
    @staticmethod  
    def get_square(self, number):  
        return number * number
```

Почему это ошибка:

Статический метод не получает `self`.

Правильно:

```
class MyMath:  
    @staticmethod  
    def get_square(number):  
        return number * number
```

6. Мини-примеры для закрепления

Пример 1. Декоратор для приветствия

```
def before_after(func):  
    def wrapper():  
        print("Начало")  
        func()  
        print("Конец")  
  
    return wrapper  
  
@before_after  
def hello():  
    print("Привет")  
  
hello()
```

Закрепляется базовая структура декоратора.

Пример 2. Декоратор проверки числа

```
def only_positive(func):  
    def wrapper(number):  
        if number > 0:  
            func(number)  
        else:  
            print("Число должно быть положительным")  
  
    return wrapper  
  
@only_positive  
def print_number(number):  
    print(number)  
  
print_number(10)  
print_number(-5)
```

Закрепляется проверка аргумента.

Пример 3. `@property`

```
class Product:
    def __init__(self, price):
        self.__price = price

    @property
    def price(self):
        return self.__price

product = Product(1000)

print(product.price)
```

Закрепляется свойство для чтения.

Пример 4. Setter

```
class Product:
    def __init__(self, price):
        self.__price = price

    @property
    def price(self):
        return self.__price

    @price.setter
    def price(self, value):
        if value >= 0:
            self.__price = value

product = Product(1000)

product.price = 800

print(product.price)
```

Закрепляется управляемое изменение значения.

Пример 5. `__str__`

```
class Product:
    def __init__(self, title, price):
        self.title = title
        self.price = price

    def __str__(self):
        return f"{self.title}: {self.price}"

product = Product("Ноутбук", 70000)

print(product)
```

Закрепляется красивый вывод объекта.

Пример 6. Статический метод

```
class TextHelper:
    @staticmethod
    def is_long(text):
        return len(text) > 10

print(TextHelper.is_long("Python"))
```

Закрепляется `@staticmethod`.

Пример 7. Классовое поле

```
class User:
    count = 0

    def __init__(self, name):
        self.name = name
        User.count = User.count + 1

user1 = User("Аня")
user2 = User("Борис")

print(User.count)
```

Закрепляется общее поле класса.

7. Практические задания

Лёгкий уровень

1. Написать декоратор, который перед вызовом функции выводит `"Функция запущена"`.
2. Написать класс `User` с приватным полем `__name` и `@property` для чтения имени.
3. Написать класс `Product` с полем цены и красивым `__str__`.
4. Написать статический метод, который проверяет, является ли число чётным.
5. Создать классовое поле `count`, которое считает количество созданных объектов.

Средний уровень

1. Написать декоратор, который проверяет, что число положительное.
2. Написать класс `User`, где возраст нельзя установить отрицательным.
3. Написать класс `Book`, который красиво выводится через `print(book)`.
4. Написать класс `Email`, где через setter нельзя установить email без символа `@`.
5. Написать класс `Counter`, который считает количество созданных счётчиков.

Повышенный уровень

1. Написать декоратор, который проверяет, что строка не пустая.
2. Написать класс `BankCard`, где баланс доступен через `@property`, а изменить его напрямую нельзя.
3. Написать класс `Password`, где пароль можно установить только если его длина не меньше 8 символов.

8. Контрольные вопросы

1. Что такое декоратор?
2. Для чего нужна функция `wrapper`?
3. Почему декоратор должен вернуть функцию?
4. Что означает запись `@decorator`?
5. Для чего нужен `@property`?
6. Чем `property` отличается от обычного метода?
7. Что такое `setter`?
8. Для чего нужен `__str__`?
9. Почему `__str__` должен возвращать строку?
10. Что такое статический метод?

11. Когда можно использовать `@staticmethod` ?

12. Что такое классовое поле?

9. Краткий конспект в конце

Главная идея темы: декораторы, `property` и магические методы позволяют управлять поведением функций и объектов более гибко.

Основные конструкции:

```
def decorator(func):  
    def wrapper():  
        func()  
  
    return wrapper
```

```
@property  
def name(self):  
    return self.__name
```

```
@name.setter  
def name(self, value):  
    self.__name = value
```

```
def __str__(self):  
    return "Текстовое описание объекта"
```

```
@staticmethod  
def helper():  
    pass
```

Важно запомнить:

- декоратор изменяет поведение функции;
- `wrapper` обычно вызывает исходную функцию;
- `@property` позволяет обращаться к методу как к полю;
- `setter` контролирует изменение свойства;
- `__str__` отвечает за красивый вывод;
- статический метод не использует `self`;
- классовое поле общее для класса.

10. Что ученик должен уметь после методички

После этой темы ученик должен уметь:

- понимать идею декоратора;
- писать простой декоратор;
- использовать `@property`;
- писать `setter`;
- объяснять отличие свойства от обычного метода;
- писать `__str__`;
- использовать `@staticmethod`;
- понимать классовые поля;
- применять эти инструменты в небольших классах.